Chapter 6 — Strings and Regular Expressions

Text is everywhere: user names, URLs (web addresses), log lines (one-line records a program writes about what it did), CSV files (comma-separated values: spreadsheet-like data as plain text), chat messages, HTML. This chapter covers how JavaScript represents text (**strings**), the methods you use every day to search, cut, change and compare it, the Unicode details that trip up even experienced developers (why `'😀'.length` is 2), and **regular expressions** — a compact pattern language for finding and transforming text.

What you'll learn

1. Creating strings with single quotes, double quotes and backticks; quotes inside strings; escape sequences.
2. Template literals: interpolation, multi-line strings, tagged templates and `String.raw`.
3. Indexing, `length`, `at()`, immutability, and what happens with out-of-range indexes.
4. String + number coercion and converting values to and from strings.
5. String primitives vs `String` objects (and auto-boxing).
6. The methods table: searching, extracting, case, whitespace, padding, splitting, replacing, comparing.
7. Unicode: code units vs code points vs grapheme clusters, normalization, `Intl.Segmenter`, well-formed strings.
8. Regular expressions: syntax, character classes, quantifiers, anchors, groups, flags, lookaround.
9. Regex methods: `test`, `exec`, `match`, `matchAll`, `replace`/`replaceAll`, `search`, `split` — and the `lastIndex` trap.
10. Modern regex features: named groups, `d`, `u` and `v` flags, `RegExp.escape`, modifiers, duplicate named groups.

Before you start. You need variables and types (Chapter 2) and functions/arrow functions (Chapter 4). Array methods such as `map` and `join` appear in examples; they are explained fully in Chapter 7.

All code is in `code/`; files are ES modules (files using `import`/`export`, which always run in strict mode — the stricter variant of JavaScript from Chapter 1) and outputs are real, from Node 24.

6.1 What a string is

What it is

A **string** is an ordered, **immutable** sequence of characters used to represent text. It is one of JavaScript's **primitive** types: `typeof 'hi'` is `'string'`. "Immutable" means no operation ever changes a string in place — every method returns a *new* string.

More precisely, a JavaScript string is a sequence of **UTF-16 code units** (16-bit numbers). For everyday English text one code unit = one character; section 6.5 explains when that stops being true.

Why it matters

Most data that enters a program arrives as text — form fields, query strings (the `?key=value&…` part of a URL), JSON, files — and must be cleaned, validated, split and reformatted. String handling bugs (trailing spaces, case differences, emoji breaking a length limit) are among the most common bugs in web apps.

6.2 Creating strings

Three kinds of quotes

Delimiter	Example	Interpolation `${}`	Real line breaks allowed	Typical use
Single quotes `'…'`	`'It works'`	No	No	Plain strings (common style default in style guides — shared formatting rulebooks — e.g. Airbnb/StandardJS)
Double quotes `"…"`	`"It works"`	No	No	Plain strings (Prettier's default); identical behaviour to single quotes
Backticks `` `…` ``	`` `Hi ${name}` ``	**Yes**	**Yes**	Anything built from values, multi-line text, tagged templates

Single and double quotes are exactly equivalent; pick one and stay consistent (a formatter — a tool that rewrites code layout automatically — such as Prettier will enforce it — see Chapter 10).

Quotes inside strings

To put a quote character inside a string, either surround the string with a *different* kind of quote or **escape** the inner quote with a backslash.

From `code/01_string_basics.mjs`:

```
console.log("She said 'hi'", '|', 'He said "bye"', '|', 'It\'s escaped', '|', `mix 'both' "kinds"`);
```

Output:

```
She said 'hi' | He said "bye" | It's escaped | mix 'both' "kinds"
```

Escape sequences

A backslash `\` inside a string starts an **escape sequence**: two or more characters in the source that stand for one special character in the value.

Escape	Produces	Notes
`\'` `\"` `\``	The quote character itself	Needed only when it matches the delimiter
`\\`	One backslash	Windows paths: `'C:\\temp'`
`\n`	Line feed (new line)	The normal line break on Linux/macOS and in most text protocols (agreed formats for messages between programs, like HTTP)
`\r`	Carriage return	Windows line endings are `\r\n`
`\t`	Horizontal tab	
`\b`	Backspace (U+0008)	Rare. Careful: inside a **regex** (regular expression, section 6.7), `\b` means "word boundary"
`\f`	Form feed (U+000C)	Rare (old printers)
`\v`	Vertical tab (U+000B)	Rare; supported by all modern engines
`\0`	Null character (U+0000)	Must not be followed by a digit (`'\01'` is a legacy octal — base-8 — escape, a `SyntaxError` in strict mode)
`\xHH`	Character with hex (hexadecimal, base-16: digits 0–9 then A–F) code HH (2 digits)	`'\x41'` is `'A'`
`\uXXXX`	UTF-16 code unit with hex code XXXX (4 digits)	`'\u00e9'` is `'é'`
`\u{X…}`	Any Unicode code point (the number Unicode gives a character, section 6.5; 1–6 hex digits), ES2015	`'\u{1F600}'` is `'😀'`
`\` + line break	Line continuation (no character added)	Old way to split a long literal across lines

```
console.log('Tab:\t|  Backslash: \\  Unicode: \u00e9 \u{1F600}  Hex: \x41');
console.log('Line 1\nLine 2');
console.log(JSON.stringify('a\r\nb\0c'), 'a\r\nb'.length);
```

Output:

```
Tab:    |  Backslash: \  Unicode: é 😀  Hex: A
Line 1
Line 2
"a\r\nb\u0000c" 4
```

`'a\r\nb'.length` is 4: each escape is a single character in the final string, however many characters it took to type.

6.3 Template literals

What it is

A **template literal** (ES2015) is a string written between backticks. It adds three abilities that quoted strings lack:

1. **Interpolation** — `${expression}` inserts the value of *any* JavaScript expression, converted to a string.
2. **Multi-line text** — real line breaks in the source become line breaks in the value.
3. **Tags** — a function name placed before the backtick receives the pieces and values and can build any result.

How it works

```
const item = 'coffee', price = 3.5, qty = 2;
console.log(`${qty} x ${item} = $${(price * qty).toFixed(2)} ${qty > 1 ? '(bulk)' : ''}`);
const multi = `Dear user,
  your order has shipped.`;
console.log(multi);
```

Output:

```
2 x coffee = $7.00 (bulk)
Dear user,
  your order has shipped.
```

- `$${…}`: the first `$` is a literal dollar sign; `${` starts interpolation.
- Any expression is allowed inside `${}`: arithmetic, method calls, ternaries (`cond ? a : b`), even nested templates.
- In a multi-line template, **indentation is part of the string** — the two spaces before "your" are kept.

Before ES2015, multi-line text needed `\n` escapes and `+` concatenation: `'This is line 1\n' + 'This is line 2'`. Templates made that obsolete.

Tagged templates and `String.raw`

A **tag** is a function called with (a) an array of the literal text pieces and (b) the interpolated values as further arguments. Libraries use tags to escape HTML, build SQL (the language for querying databases) safely, or define CSS (styled-components' `styled.div`…``, GraphQL's `gql`…`` (GraphQL is a query language for APIs), lit-html's `html`…``).

```
function highlight(strings, ...values) {
  return strings.reduce((out, s, i) => out + s + (i < values.length ? `[${values[i]}]` : ''), '');
}
console.log(highlight`Hello ${'Ana'}, you have ${3} messages`);
console.log(String.raw`C:\new\table`, '|', 'C:\new\table'.length, String.raw`C:\new\table`.length);
```

Output:

```
Hello [Ana], you have [3] messages
C:\new\table | 10 12
```

`strings` is `['Hello ', ', you have ', ' messages']` and `values` is `['Ana', 3]`: there is always one more string piece than values. The built-in tag `String.raw` returns the text *without* processing escapes — `\n` stays a backslash and an `n` — which is handy for Windows paths and regex sources. The normal string `'C:\new\table'` has only 10 characters because `\n` and `\t` became a newline and a tab.

Common mistakes

- Using `'${name}'` (normal quotes) and wondering why nothing is substituted — only backticks interpolate.

- Building HTML with templates from user input without escaping → **XSS** (cross-site scripting). Escape or use `textContent` (Chapter 14).
- Building SQL by interpolation → SQL injection (an attacker's input gets run as part of your database command). Use parameterised queries (the database receives the command and the values separately, so values are never treated as code).

6.4 Working with strings: indexing, immutability, coercion, String objects

Indexing, `length` and `at()`

Characters are numbered from **0**. Read one with bracket notation `str[i]`, `str.charAt(i)`, or `str.at(i)` (ES2022), which also accepts **negative** indexes counting from the end.

```
const greeting = 'Hello, World!';
console.log(greeting.length, greeting[0], greeting[7], greeting.at(-1), greeting[50],
greeting.charAt(50) === '');
```

Output:

```
13 H W ! undefined true
```

- `length` counts code units, including spaces and punctuation: `'Hello, World!'` has 13.
- The first character is at `0`; the last is at `length - 1` (or `at(-1)`).
- An index past the end gives `undefined` with brackets, and an empty string `''` with `charAt`.

Strings are immutable

```
const s = 'cat';
try { s[0] = 'b'; } catch (e) { console.log('assigning s[0] ->', e.constructor.name); }
console.log(s, 'b' + s.slice(1));
```

Output:

```
assigning s[0] -> TypeError
cat bat
```

Assigning to an index never changes a string. In **sloppy** (non-strict) scripts the assignment is silently ignored; in **strict** mode (modules, classes) it throws a `TypeError`. To "change" a string, build a new one.

Concatenation and coercion

The `+` operator means **addition** for two numbers but **concatenation** as soon as either side is a string — the other side is converted to a string first. Other arithmetic operators (`-`, `*`, `/`) convert strings to numbers instead.

```
console.log('The number is ' + 5, '|', '5' + 3, '5' - 3, '5' * '2', 1 + 2 + '3', '1' + 2 + 3);
console.log(String(42), (42).toString(), `${null} ${undefined}`, String([1, 2]), String({}));
```

Output:

```
The number is 5 | 53 2 10 33 123
42 42 null undefined 1,2 [object Object]
```

`+` is evaluated left to right: `1 + 2 + '3'` is `3 + '3'` = `'33'`, but `'1' + 2 + 3` is `'12' + 3` = `'123'`. To convert **to** a string use `String(x)` (safe for `null`/`undefined`) or a template; `x.toString()` throws on `null`/`undefined`. To convert **from** a string see the `Number`/`parseInt` rows in section 6.6. The complete coercion table is in Chapter 2.

String primitives vs `String` objects

JavaScript also has a `String` **wrapper object**, created with `new String('…')`. You essentially never want it.

```
const obj = new String('Hello');
const prim = 'Hello';
console.log(typeof obj, typeof prim, obj === prim, obj == prim, obj.valueOf() === prim);
console.log(new String('a') === new String('a'), typeof String(123));
console.log(prim.toUpperCase(), 'methods work on primitives via temporary wrapper objects');
```

Output:

```
object string false true true
false string
HELLO methods work on primitives via temporary wrapper objects
```

	String primitive `'Hello'`	**String object `new String('Hello')`**
`typeof`	`'string'`	`'object'`
Equality	By value — two equal primitives are `===`	By reference — two wrappers are never `===`, even with the same text
Truthiness	`''` is falsy	Every object is truthy — `new String('')` is truthy!
Cost	Cheap; engines optimise heavily	Extra allocation (reserving memory for a new object)
Methods	Available through **auto-boxing**	Available directly

Auto-boxing: when you call `'Hello'.toUpperCase()`, the engine behaves as if it briefly wrapped the primitive in a `String` object to find the method on `String.prototype` (the shared object where all strings' methods live — prototypes are explained in Chapter 9), then throws the wrapper away. That is why primitives "have methods" without being objects. `String(123)` **without** `new` is a conversion function and returns a primitive.

Comparing strings

```
console.log('apple' < 'banana', 'Zebra' < 'apple', 'a'.localeCompare('B'), ['é', 'z', 'e'].sort(),
['é', 'z', 'e'].sort((a, b) => a.localeCompare(b)));
```

Output:

```
true true -1 [ 'e', 'z', 'é' ] [ 'e', 'é', 'z' ]
```

`<`, `>` and the default `sort()` compare **code unit by code unit**. All uppercase letters come before all lowercase ones (`'Z'` is 90, `'a'` is 97), and accented letters sort after `z`. For human ordering use

`localeCompare` or an `Intl.Collator` (section 6.6; `Intl` is the built-in internationalisation namespace for language-aware formatting and sorting).

Self-check

1. `'abc'[-1]` vs `'abc'.at(-1)`? → `undefined` vs `'c'`.
2. `'3' + 4 + 5`? → `'345'`. And `3 + 4 + '5'`? → `'75'`.
3. Is `new String('') ? 'y' : 'n'` `'y'` or `'n'`? → `'y'` — objects are truthy.
4. Why is `'B' < 'a'` true? → Code unit 66 < 97.

6.5 Unicode: code units, code points and graphemes

The problem it solves

Why is the length of `'😀'` 2? Why can reversing a string produce garbage? Why are `'café'` and `'café'` sometimes not equal? The answers are all about how text is encoded.

Three levels of "character"

Level	What it is	Example	JavaScript view
Code unit	One 16-bit number in UTF-16	`'😀'` = two units `D83D DE00`	`length`, `str[i]`, `charCodeAt`, `slice` indexes
Code point	One Unicode character number (U+0000 … U+10FFFF)	`'😀'` = U+1F600	`codePointAt`, `String.fromCodePoint`, `for…of`, `[...str]`
Grapheme cluster	What a person sees as one character	`'👨‍👩‍👧'` = 5 code points, 1 grapheme	`Intl.Segmenter`

Code points up to U+FFFF (the **Basic Multilingual Plane**, which covers most living scripts) fit in one code unit. Code points above it — most emoji, rare CJK (Chinese, Japanese, Korean) characters, musical and math symbols — are stored as a **surrogate pair**: two code units in the ranges D800–DBFF ("high") then DC00–DFFF ("low"). A lone half of a pair is a **lone surrogate**, and a string containing one is **not well-formed**.

From `code/03_unicode.mjs`:

```
const accented = 'café';                    // é as one code point U+00E9
const decomposed = 'cafe\u0301';            // e + combining acute accent U+0301
console.log(accented, decomposed, accented === decomposed, accented.length, decomposed.length);
console.log(accented === decomposed.normalize('NFC'), accented.normalize('NFD').length);

const smile = '😀';
console.log(smile.length, smile.charCodeAt(0).toString(16), smile.charCodeAt(1).toString(16));
console.log(smile.codePointAt(0).toString(16), String.fromCodePoint(0x1f600) === smile, '\u{1F600}'
=== smile);
console.log(JSON.stringify(smile[0]), smile[0].isWellFormed(), smile.isWellFormed());

const word = 'hi😀!';
console.log(word.length, [...word].length, [...word]);
console.log('naive reverse :', word.split('').reverse().join('').isWellFormed());
console.log('safer reverse :', [...word].reverse().join(''));
```

Output:

```
café café false 4 5
true 5
2 d83d de00
1f600 true true
"\ud83d" false true
5 4 [ 'h', 'i', '😀', '!' ]
naive reverse : false
safer reverse : !😀ih
```

What this shows:

1. **Normalization.** "é" can be one code point (U+00E9) or two (`e` + combining accent U+0301). They look identical but are not `===`. `str.normalize('NFC')` composes to the short form; `'NFD'` decomposes. Normalize user input before comparing or storing it (e.g. usernames). `'NFKC'`/`'NFKD'` additionally fold "compatibility" variants such as `ﬁ` → `fi`.
2. **Surrogate pairs.** `'😀'.length` is 2, and `smile[0]` is a lone surrogate. `codePointAt` / `fromCodePoint` / `\u{…}` work with whole code points.
3. **Iteration is by code point.** `for…of`, spread `[...str]` and `Array.from(str)` split correctly into code points; `split('')` splits into code units and can cut pairs in half, producing a broken string when reversed.

Grapheme clusters and `Intl.Segmenter`

Some visible characters are several code points joined together: family emoji (joined with the invisible **zero-width joiner** U+200D), flags (two "regional indicator" letters), skin-tone variants, letters plus combining accents. Even code-point iteration splits these. `Intl.Segmenter` (in all modern browsers and Node) splits text into graphemes, words or sentences according to Unicode rules:

```
const family = '👨‍👩‍👧';
const flag = '🇮🇳';
console.log(family.length, [...family].length, flag.length, [...flag].length);
const seg = new Intl.Segmenter('en', { granularity: 'grapheme' });
const graphemes = (s) => [...seg.segment(s)].map((x) => x.segment);
console.log(graphemes(`a${family}b${flag}`).length, graphemes('cafe\u0301').length);

const words = [...new Intl.Segmenter('en', { granularity: 'word' }).segment('Hello, big world!')]
  .filter((x) => x.isWordLike).map((x) => x.segment);
console.log(words);
```

Output:

```
8 5 4 2
4 4
[ 'Hello', 'big', 'world' ]
```

The family emoji is 8 code units, 5 code points, but **1** grapheme. Use grapheme counts for user-facing limits ("max 280 characters"), cursor movement and truncation.

Well-formed strings and byte length

```
const broken = 'ab\uD800';
console.log(broken.isWellFormed(), broken.toWellFormed().codePointAt(2).toString(16));
console.log(encodeURIComponent(broken.toWellFormed()));
try { encodeURIComponent(broken); } catch (e) { console.log('encodeURIComponent(lone surrogate) ->',
e.constructor.name); }

const enc = new TextEncoder();
for (const s of ['abc', 'café', '😀']) console.log(s, 'length', s.length, 'utf8 bytes',
enc.encode(s).length);
```

Output:

```
false fffd
ab%EF%BF%BD
encodeURIComponent(lone surrogate) -> URIError
abc length 3 utf8 bytes 3
café length 4 utf8 bytes 5
😀 length 2 utf8 bytes 4
```

- `isWellFormed()` / `toWellFormed()` (ES2024) detect and repair lone surrogates (replacing each with U+FFFD, the "replacement character" `�`). APIs that must encode text — `encodeURIComponent` (converts text into the `%XX` form safe inside a URL), `TextEncoder`, `fetch` bodies — either throw or silently substitute on broken strings.
- Files, network payloads (the data carried in a message) and database columns are usually measured in **UTF-8 bytes**, not in `length`. (A **byte** is 8 bits; **UTF-8** is the most common text encoding on the web, using 1–4 bytes per character.) Use `new TextEncoder().encode(s).length` (browser) or `Buffer.byteLength(s)` (Node) when a byte limit matters.

In the interview

30-second answer: "JS strings are UTF-16 code units. `length` and indexing count units, so characters beyond U+FFFF like most emoji count as 2. `for…of` and spread iterate code points; `Intl.Segmenter` gives user-perceived characters. Normalize with `normalize('NFC')` before comparing text."

- *How do you reverse a string safely?* `[...s].reverse().join('')` is correct for code points; for combining marks and ZWJ emoji, reverse the graphemes from `Intl.Segmenter`.
- *How do you count characters for a tweet-like limit?* Count graphemes, not `length`.

Self-check

1. `'😀'.length`? → 2.
2. `[...'😀'].length`? → 1.
3. Why might two identical-looking strings not be equal? → Different normalization forms (composed vs decomposed).
4. Which API gives a word list for text in any language? → `Intl.Segmenter` with `granularity: 'word'`.

6.6 String methods

How to read this section

Every string method **returns a new value** and leaves the original untouched. The table groups them by job; the code afterwards shows each in action. "Index" arguments are code-unit positions; an `end` index is **exclusive** (the character at `end` is not included).

Job	Method	Returns	Notes
Size	`length` (property)	number of code units	not a method — no `()`
Read a char	`charAt(i)`	1-char string, or `''` if out of range	
	`at(i)` (ES2022)	1-char string or `undefined`; negative `i` counts from end	
	`charCodeAt(i)`	UTF-16 code unit number 0–65535	`codePointAt(i)` for full code points
	`String.fromCharCode(...codes)` / `String.fromCodePoint(...cps)`	new string	**static**: called on `String`, not on a string
Search	`indexOf(sub, from?)` / `lastIndexOf(sub, from?)`	index or **-1**	case-sensitive
	`includes(sub, from?)`	boolean	ES2015
	`startsWith(sub, pos?)` / `endsWith(sub, len?)`	boolean	ES2015
	`search(regex)`	index of first regex match or -1	
	`match(regex)` / `matchAll(regex)`	match info / iterator (an object that hands out results one at a time, Chapter 8)	section 6.8
Extract	`slice(start, end?)`	substring; **negative** indexes count from end	the one to use
	`substring(start, end?)`	substring; negatives become 0; swaps args if start > end	older
	`substr(start, length?)`	substring by **length**	legacy (Annex B, the spec's appendix of old web-compatibility features, "normative optional") — avoid
Case	`toUpperCase()` / `toLowerCase()`	new string	locale-independent Unicode rules
	`toLocaleUpperCase(loc?)` / `toLocaleLowerCase(loc?)`	new string	locale-specific rules (a locale is a language + region setting such as `tr-TR`; e.g. Turkish dotted İ)
Whitespace	`trim()` / `trimStart()` / `trimEnd()`	new string	removes spaces, tabs, newlines and other Unicode whitespace
Padding	`padStart(len, fill?)` / `padEnd(len, fill?)`	new string of at least `len`	ES2017
Build	`repeat(n)`	`n` copies joined	
	`concat(...parts)`	joined string	`+` or templates are more common

Job	Method	Returns	Notes
Split / join	`split(sep?, limit?)`	**array** of pieces	`sep` can be a string or regex; `arr.join(sep)` reverses it
Replace	`replace(pattern, replacement)`	new string	string pattern → first occurrence only
	`replaceAll(pattern, replacement)` (ES2021)	new string	every occurrence; a regex must have `g`
Compare	`localeCompare(other, locale?, options?)`	negative / 0 / positive	language-aware ordering
Unicode	`normalize(form?)`, `isWellFormed()`, `toWellFormed()`	string / boolean	section 6.5
Convert	`toString()` / `valueOf()`	the primitive string	mainly for `String` objects

Searching

From `code/02_string_methods.mjs`:

```
const log = 'ERROR 42: disk full; ERROR 7: fan';
console.log(log.indexOf('ERROR'), log.lastIndexOf('ERROR'), log.indexOf('WARN'), log.indexOf('ERROR',
1));
console.log(log.includes('disk'), log.startsWith('ERROR'), log.endsWith('fan'), log.startsWith('42',
6));
console.log(log.search(/\d+/));
```

Output:

```
0 21 -1 21
true true true true
6
```

Use `includes` when you only need yes/no; `indexOf(...) !== -1` is the older idiom. A common bug is `if (s.indexOf(x))` — that is falsy when `x` is found **at index 0** and truthy (−1) when it is missing.

Extracting: `slice` vs `substring` vs `substr`

```
const text = 'Hello, World!';
console.log(text.slice(0, 5), '|', text.slice(7), '|', text.slice(-6, -1));
console.log(text.substring(0, 5), '|', text.substring(5, 0), '|', text.substring(-3, 2));
console.log(text.slice(5, 0) === '', text.substr(7, 5), '(substr: legacy, avoid)');
console.log(text.charAt(4), text.at(-1), text.charCodeAt(0), String.fromCharCode(72, 105));
```

Output:

```
Hello | World! | World
Hello | Hello | He
true World (substr: legacy, avoid)
o ! 72 Hi
```

- `slice(0, 5)` takes indexes 0–4 → `'Hello'`; the end index 5 (the comma) is excluded. Omit `end` to go to the end.

- `slice` understands negative indexes (`-6` = 6 from the end); `substring` turns them into 0 and swaps reversed arguments; `slice` with start > end returns `''`.
- `substr(7, 5)` means "5 characters starting at 7". It's kept only for old code — use `slice`.

Case, whitespace and padding

```
console.log(text.toUpperCase(), text.toLowerCase());
console.log('istanbul'.toLocaleUpperCase('tr-TR'), 'istanbul'.toUpperCase());
console.log('Straße'.toUpperCase(), 'Dž'.toLowerCase());

const raw = '   padded text \n';
console.log(JSON.stringify(raw.trim()), JSON.stringify(raw.trimStart()),
JSON.stringify(raw.trimEnd()));
console.log('7'.padStart(3, '0'), 'ab'.padEnd(5, '.') + '|', '4111222233334444'.slice(-4).padStart(16,
'*'));
```

Output:

```
HELLO, WORLD! hello, world!
İSTANBUL ISTANBUL
STRASSE Dž
"padded text" "padded text \n" "   padded text"
007 ab...| ************4444
```

- Case conversion is Unicode-aware and can change length (`ß` → `SS`). Turkish has a dotted capital `İ`, which only the locale-aware version produces.
- For **case-insensitive comparison**, lowercasing both sides works for simple text; for accents and other languages prefer `a.localeCompare(b, undefined, { sensitivity: 'base' }) === 0`.
- `padStart` is perfect for zero-padding (`'7'` → `'007'`) and masking card numbers.

Building, splitting and replacing

```
console.log('ab'.repeat(3), 'a'.concat('b', 'c', 1), ['x', 'y', 'z'].join('-'));
console.log('a,b,,c'.split(','), 'hello'.split(''), 'a b  c'.split(' '), 'a b  c'.split(/\s+/),
'x'.split(','), 'a,b,c'.split(',', 2));
const s = 'cat, cat, cat';
console.log(s.replace('cat', 'dog'), '|', s.replaceAll('cat', 'dog'), '|', s.replace(/cat/g, 'cow'));
```

Output:

```
ababab abc1 x-y-z
[ 'a', 'b', '', 'c' ] [ 'h', 'e', 'l', 'l', 'o' ] [ 'a', 'b', '', 'c' ] [ 'a', 'b', 'c' ] [ 'x' ] [
'a', 'b' ]
dog, cat, cat | dog, dog, dog | cow, cow, cow
```

- `split` keeps empty strings between consecutive separators (`'a,b,,c'` → 4 pieces); a regex separator like `/\s+/` collapses runs of whitespace. With no match you get a one-element array; the optional second argument limits how many pieces are returned.
- **`replace` with a string pattern changes only the first occurrence** — the source of countless bugs. Use `replaceAll` or a regex with the `g` flag.

Comparing with `localeCompare` and `Intl.Collator`

```
console.log('a'.localeCompare('b'), 'b'.localeCompare('a'), 'a'.localeCompare('a'));
console.log('résumé'.localeCompare('resume', 'en', { sensitivity: 'base' }), 'ä'.localeCompare('z',
'de'), 'ä'.localeCompare('z', 'sv'));
const collator = new Intl.Collator('en', { numeric: true });
console.log(['file10', 'file2', 'file1'].sort(), ['file10', 'file2', 'file1'].sort(collator.compare));
```

Output:

```
-1 1 0
0 -1 1
[ 'file1', 'file10', 'file2' ] [ 'file1', 'file2', 'file10' ]
```

`localeCompare` returns a negative number, 0, or a positive number — exactly what `sort` expects from a comparator (a function that says which of two items comes first). Order depends on the **language**: in German `ä` sorts with `a` (before `z`); in Swedish it's a separate letter after `z`. `sensitivity: 'base'` ignores accents and case. `numeric: true` gives **natural sort** (`file2` before `file10`). When sorting many strings, create one `Intl.Collator` and reuse its `compare` — it is much faster than calling `localeCompare` repeatedly.

Converting strings to numbers

```
const wrapped = new String('boxed');
console.log(wrapped.toString(), wrapped.valueOf(), typeof wrapped.valueOf());
console.log(Number('42px'), parseInt('42px', 10), parseFloat('3.14abc'), Number(''), Number('  7 '));
```

Output:

```
boxed boxed string
NaN 42 3.14 0 7
```

`Number(str)` is strict — the whole string (ignoring surrounding whitespace) must be numeric, and `''` becomes `0`. `parseInt` / `parseFloat` read as many digits as they can and stop. Always pass the radix (the number base, e.g. 10 for decimal) to `parseInt` (`parseInt(s, 10)`).

Two real tasks

```
const slugify = (title) => title.trim().toLowerCase()
  .normalize('NFKD').replace(/[\u0300-\u036f]/g, '')   // strip accents
  .replace(/[^a-z0-9]+/g, '-').replace(/^-+|-+$/g, '');
console.log(slugify('  Crème Brûlée: 10 Tips & Tricks!  '));

const titleCase = (str) => str.split(' ').map((w) => w.charAt(0).toUpperCase() +
w.slice(1).toLowerCase()).join(' ');
console.log(titleCase('the qUICK brown fox'));
```

Output:

```
creme-brulee-10-tips-tricks
The Quick Brown Fox
```

`slugify` builds URL slugs (the readable last part of a web address, like `my-first-post`) the way blogs and CMSs (content management systems such as WordPress) do: trim, lowercase, decompose

accented letters and delete the combining marks (range U+0300–U+036F), turn every run of non-alphanumerics into `-`, then strip leading/trailing dashes.

Performance note

Because strings are immutable, `s += piece` in a loop *conceptually* creates a new string each time. Modern engines optimise this with internal "rope" structures (a tree of string pieces that is only joined into one string when needed), so it's fine for normal sizes; for building very large output, collecting pieces in an array and calling `join('')` once is a safe, predictable alternative.

Common mistakes

- `replace('a', 'b')` expecting every `a` to change.
- `if (str.indexOf(x))` instead of `includes`.
- Comparing user-entered text without `trim()` and case/accent normalization.
- Using `substr` in new code, or confusing `substring(start, end)` with `substr(start, length)`.
- Sorting names with the default `sort()` (uppercase before lowercase, accents last).

In the interview

30-second answer: "Strings are immutable primitives; methods return new strings. I use `slice` for extraction (supports negatives), `includes` / `startsWith` for tests, `split` / `join` to convert to arrays, `replaceAll` or a `/g` regex to replace everywhere, `padStart` for formatting, and `localeCompare` / `Intl.Collator` for human sorting."

- *Check whether a string is a palindrome* (reads the same backwards, like "level"). `const t = s.toLowerCase().replace(/[^a-z0-9]/g, ''); t === [...t].reverse().join('')`.
- *Count occurrences of a substring.* `s.split(sub).length - 1`, or `s.match(/sub/g)?.length ?? 0` (escape the pattern if it comes from input — `RegExp.escape`, section 6.9).
- *Difference between `slice` and `substring`?* Negative indexes and argument swapping.

Self-check

1. `'JavaScript'.slice(-6)`? → `'Script'`.
2. `'a-b-c'.split('-', 2)`? → `['a', 'b']`.
3. `'5'.padStart(3, '0')`? → `'005'`.
4. Sort `['b', 'A', 'a']` case-insensitively. → `.sort((x, y) => x.localeCompare(y, undefined, { sensitivity: 'base' }))`.

6.7 Regular expressions: the pattern language

What it is

A **regular expression** (regex, regexp) is a small pattern that describes a set of strings — "five digits", "a word followed by `@`", "anything between `<` and `>`". JavaScript regexes are `RegExp` objects. You write them in two ways:

- **Literal** `/pattern/flags` — compiled (turned into the engine's internal matching program) once when the script loads; use when the pattern is fixed.
- **Constructor** `new RegExp('pattern', 'flags')` — built from a string at run time; use when the pattern depends on data. Backslashes must be doubled inside the string: `new RegExp('\\d+')`.

Think of a regex as a tiny program that the regex engine runs against your text, trying positions from left to right until the pattern fits.

Why it matters

Validating form input, extracting fields from logs, search-and-replace across files, routing URLs (`/users/:id` becomes a regex inside routers — the code that decides which handler serves which address), syntax highlighting, tokenizers (programs that chop text into meaningful pieces such as words, numbers and symbols) — all use regexes. They are also easy to get wrong, so knowing the core syntax well pays off.

Syntax cheat sheet

Category	Syntax	Matches
Literal characters	`abc`	the text `abc`
Any character	`.`	any char except line terminators (unless the `s` flag)
Escaping	`\.` `\*` `\\` `\/`	the special character itself
Character class	`[abc]`, `[a-z0-9]`	one char from the set / range
Negated class	`[^abc]`	one char **not** in the set
Shorthand classes	`\d` `\w` `\s`	digit `[0-9]`; word char `[A-Za-z0-9_]`; whitespace
Negated shorthands	`\D` `\W` `\S`	the complements
Unicode property	`\p{L}`, `\p{Nd}`, `\p{Script=Greek}` (needs `u` or `v`)	any letter / decimal digit / Greek char
Quantifiers	`*` `+` `?`	0 or more / 1 or more / 0 or 1
Counted	`{n}` `{n,}` `{n,m}`	exactly n / at least n / n to m
Lazy quantifiers	`*?` `+?` `??` `{n,m}?`	as **few** as possible
Anchors	`^` `$`	start / end of input (of each line with `m`)
Word boundary	`\b` `\B`	position between `\w` and `\W` / not a boundary
Alternation	`a	b`	
Capturing group	`( … )`	groups and remembers the match as `$1`, `m[1]`, …
Named group	`(?<name> … )`	remembered as `groups.name` / `$<name>`
Non-capturing group	`(?: … )`	groups without remembering
Backreference	`\1`, `\k<name>`	the same text a group matched earlier
Lookahead	`(?= … )` / `(?! … )`	position followed / not followed by …
Lookbehind (ES2018)	`(?<= … )` / `(?<! … )`	position preceded / not preceded by …

Flags

Flag	Name	Effect
`g`	global	Find **all** matches (affects `match`, `replace`, `matchAll`; makes `test`/`exec` stateful — they remember something between calls)
`i`	ignoreCase	Case-insensitive
`m`	multiline	`^` and `$` match at each line start/end
`s`	dotAll (ES2018)	`.` also matches newlines
`u`	unicode (ES2015)	Work in code points; enables `\p{…}` and `\u{…}`; stricter syntax
`v`	unicodeSets (ES2024)	Superset of `u`: set operations in classes, string properties; use `u` **or** `v`, not both
`y`	sticky	Match only at exactly `lastIndex` (used by tokenizers)
`d`	hasIndices (ES2022)	Add start/end `indices` for each group

6.8 Regex methods and techniques

Which method to use

Goal	Use	Returns
Does it match?	`regex.test(str)`	`true` / `false`
First match with details	`str.match(regexWithoutG)` or `regex.exec(str)`	match array (`[0]` full match, `[1]…` groups, `.index`, `.groups`) or `null`
All matched strings	`str.match(regexWithG)`	array of strings or `null`
All matches with details	`str.matchAll(regexWithG)` (ES2020)	iterator of match arrays (throws without `g`)
Position of first match	`str.search(regex)`	index or -1
Replace	`str.replace(regex, replacement)` / `replaceAll`	new string
Split	`str.split(regex)`	array

`test`, `match`, `matchAll` and named groups

From `code/04_regex_basics.mjs`:

```
const literal = /colou?r/i;
const dynamic = new RegExp('colou?r', 'i');
console.log(literal.test('Nice COLOR'), dynamic.source, dynamic.flags, literal.toString());

const zip = /^\d{5}(-\d{4})?$/;
console.log(['12345', '12345-6789', '1234', '12345-'].map((z) => `${z}:${zip.test(z)}`).join(' '));

const text = 'Order #123 shipped, order #456 pending';
const first = text.match(/#(\d+)/);
console.log(first[0], first[1], first.index);
console.log(text.match(/#\d+/g), 'no match ->', text.match(/#x/g));

for (const m of text.matchAll(/#(?<id>\d+) (?<status>\w+)/g)) {
  console.log(m.index, m.groups.id, m.groups.status);
}

const { groups: { year, month, day } } = '2024-07-04'.match(/(?<year>\d{4})-(?<month>\d{2})-(?
<day>\d{2})/);
console.log(year, month, day);
```

Output:

```
true colou?r i /colou?r/i
12345:true 12345-6789:true 1234:false 12345-:false
#123 123 6
[ '#123', '#456' ] no match -> null
6 123 shipped
26 456 pending
2024 07 04
```

- `u?` makes the `u` optional, so both spellings match; the `i` flag ignores case.
- The ZIP pattern is **anchored** with `^…$` — without anchors, `test` would accept `'abc12345xyz'` because a match *anywhere* counts. Validation regexes almost always need anchors.
- `match` without `g` gives the full match, groups and position; with `g` it gives only the matched strings. Watch out: **no match returns** `null`, not an empty array — `str.match(re).length` crashes. Use `str.match(re) ?? []` or `matchAll`.
- **Named groups** `(?<year>…)` make results self-documenting and combine nicely with destructuring.

`replace` with patterns and functions

```
console.log('2024-07-04'.replace(/(\d{4})-(\d{2})-(\d{2})/, '$3/$2/$1'));
console.log('2024-07-04'.replace(/(?<y>\d{4})-(?<m>\d{2})-(?<d>\d{2})/, '$<d>.$<m>.$<y>'));
console.log('price: 10 USD, 25 USD'.replace(/(\d+) USD/g, (all, n) => `€${(n * 0.9).toFixed(2)}`));
console.log('snake_case_name'.replace(/_(\w)/g, (_, c) => c.toUpperCase()));
console.log('a.b.c'.replaceAll('.', '/'), 'x'.replace('x', '$&$&'));
console.log('one, two;three  four'.split(/[,;\s]+/), '1+2-3'.split(/([+-])/));
```

Output:

```
04/07/2024
04.07.2024
price: €9.00, €22.50
snakeCaseName
a/b/c xx
[ 'one', 'two', 'three', 'four' ] [ '1', '+', '2', '-', '3' ]
```

Special patterns inside a replacement **string**: `$1`…`$99` (numbered group), `$<name>` (named group), `$&` (whole match), `$`` / `$'` (text before / after the match), `$$` (a literal `$`). They apply even when the search pattern is a plain string — so a replacement containing user text should be passed through a **function** instead. A replacement **function** receives `(match, group1, group2, …, offset, fullString, groups)` and returns the text to insert — the most powerful form. If you split with a regex containing a **capturing group**, the captured separators are included in the result.

Greedy vs lazy, anchors, flags in action

```
const html = '<b>bold</b> and <i>italic</i>';
console.log(html.match(/<.+>/)[0], '|', html.match(/<.+?>/)[0], '|', html.match(/<[^>]+>/g));

console.log(/\bcat\b/.test('concatenate'), /\bcat\b/.test('a cat sat'));
const lines = 'id: 1\nname: Bo\nid: 2';
console.log(lines.match(/^id: \d+$/g), lines.match(/^id: \d+$/gm));

console.log(/start.+end/.test('start\nend'), /start.+end/s.test('start\nend'));
```

Output:

```
<b>bold</b> and <i>italic</i> | <b> | [ '<b>', '</b>', '<i>', '</i>' ]
false true
null [ 'id: 1', 'id: 2' ]
false true
```

- Quantifiers are **greedy** by default: `.+` grabs as much as it can and then backs off, so `<.+>` spans from the first `<` to the *last* `>`. Adding `?` makes it **lazy**. A negated class (`[^>]+`) is usually clearer and faster than a lazy dot.
- `\b` stops `cat` from matching inside `concatenate`.
- Without `m`, `^`/`$` refer to the whole input, so a multi-line text matches nothing; with `m` they work per line.
- `.` doesn't match `\n` unless you add the `s` flag.

(Parsing real HTML with regexes is fragile — use `DOMParser` in the browser or a parser library; a **parser** reads text in a known format and builds a structured representation of it. The example only illustrates greediness.)

Lookaround and backreferences

```
console.log('$30 and €40 and $5'.match(/(?<=\$)\d+/g));        // numbers preceded by $
console.log('100px 2em 30px'.match(/\d+(?=px)/g));             // numbers followed by px
const strong = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}$/;
console.log(['weakpass', 'Str0ngPass', 'Short1A'].map((p) => `${p}:${strong.test(p)}`).join(' '));
console.log('this is is a test test'.match(/\b(\w+) \1\b/g));
```

Output:

```
[ '30', '5' ]
[ '100', '30' ]
weakpass:false Str0ngPass:true Short1A:false
[ 'is is', 'test test' ]
```

Lookarounds check what is before or after a position *without including it* in the match. Stacking lookaheads at the start (`(?=.*[a-z])(?=.*[A-Z])(?=.*\d)`) expresses "must contain each of these" — the classic password-rule pattern. The backreference `\1` means "the same text group 1 captured", which finds doubled words.

The `lastIndex` trap and `exec` loops

```
const g = /a/g;
console.log(g.test('a'), g.lastIndex, g.test('a'), g.lastIndex, g.test('a'));

const re = /\d+/g; let m; const found = [];
while ((m = re.exec('a1b22c333')) !== null) found.push(`${m[0]}@${m.index}`);
console.log(found);

const sticky = /\d+/y; sticky.lastIndex = 1;
console.log(sticky.exec('a12b')?.[0], (sticky.lastIndex = 0, sticky.exec('a12b')));
```

Output:

```
true 1 false 0 true
[ '1@1', '22@3', '333@6' ]
12 null
```

A regex with `g` (or `y`) remembers where its last match ended in its **`lastIndex`** property, and `test`/`exec` start searching from there. Reusing one global regex to `test` several strings therefore gives alternating `true`/`false` results — a notorious bug when a `/g` regex is stored in a constant and used for validation. Fixes: don't use `g` for `test`, or reset `lastIndex = 0`, or use `matchAll` (which copies the regex internally). The `while (exec)` loop is the pre-2020 way to walk all matches with details; `matchAll` replaces it. The **`sticky`** `y` flag only matches starting exactly at `lastIndex` — a building block for tokenizers.

6.9 Modern regex features and practical recipes

From `code/05_regex_modern.mjs`:

```
console.log(/^.$/.test('😀'), /^.$/u.test('😀'));
console.log('Ünïcödé 123 ñ'.match(/\p{L}+/gu));
console.log('Price: ৳৫ or 34'.match(/\p{Nd}+/gu));
console.log('Hi 👋 there 🎉'.replace(/\p{Extended_Pictographic}/gu, '*'));

console.log('abcdef'.match(/[\p{L}--[aeiou]]/gv));
console.log('I 🏴 flags'.match(/\p{RGI_Emoji_Flag_Sequence}/v)?.[0]);

const userInput = 'a.b*c';
const safe = new RegExp(RegExp.escape(userInput), 'g');
console.log(RegExp.escape(userInput), 'xa.b*cx axbbc'.replace(safe, '[hit]'));
console.log('without escaping:', 'xa.b*cx axbbc'.replace(new RegExp(userInput, 'g'), '[hit]'));

const withIdx = /(?<key>\w+)=(?<val>\w+)/d.exec('  mode=dark');
console.log(withIdx.indices.groups.key, withIdx.indices.groups.val);

const dateRe = /(?<y>\d{4})-(?<m>\d{2})|(?<m>\d{2})\/(?<y>\d{4})/;
console.log('2024-07'.match(dateRe).groups, '07/2024'.match(dateRe).groups);

console.log(/^(?i:hello) World$/.test('HELLO World'), /^(?i:hello) World$/.test('HELLO WORLD'));
```

Output:

```
false true
[ 'Ünïcödé', 'ñ' ]
[ '𝟑𝟒', '34' ]
Hi * there *
[ 'b', 'c', 'd', 'f' ]
🇮🇳
\x61\.b\*c x[hit]x axbbc
without escaping: xa.b*cx [hit]
[ 2, 6 ] [ 7, 11 ]
[Object: null prototype] { y: '2024', m: '07' } [Object: null prototype] { y: '2024', m: '07' }
true false
```

Feature	Version	What it gives you
`u` flag + `\p{…}` property escapes	ES2015 / ES2018	Code-point matching; letters, digits, emoji in **any** script (`\w` and `\d` are ASCII-only)
`v` flag	ES2024	Class **set operations** — difference `--`, intersection `&&`, nested classes — and multi-code-point properties like `\p{RGI_Emoji_Flag_Sequence}`
`d` flag	ES2022	`indices` with `[start, end]` for the match and each group — for editors and highlighters
`RegExp.escape(str)`	**ES2025**	Escapes every regex-special character so user text is matched literally. Note it also hex-escapes a leading letter or digit (`\x61`) so the result is safe even when placed after another escape
Duplicate named groups	**ES2025**	The same group name may appear in different `
Pattern modifiers `(?i:…)`	**ES2025**	Turn `i`, `m` or `s` on/off for part of a pattern

All of these run in Node 24 and current evergreen browsers (browsers that update themselves automatically). Without `RegExp.escape`, the user's `.` and `*` were treated as regex operators and matched the wrong text — an injection bug, and with patterns like `(a+)+` a potential **ReDoS** (regular-expression denial of service: a pattern that takes exponential time on certain inputs and freezes the thread).

Practical recipes

```
const logs = `127.0.0.1 - [10/Oct/2024:13:55:36] "GET /index.html HTTP/1.1" 200 2326
10.0.0.5 - [10/Oct/2024:13:56:01] "POST /api/login HTTP/1.1" 401 87`;
const lineRe = /^(?<ip>\S+) - \[(?<time>[^\]]+)\] "(?<method>[A-Z]+) (?<path>\S+) [^"]+" (?
<status>\d{3}) (?<bytes>\d+)$/gm;
console.table([...logs.matchAll(lineRe)].map((m) => ({ ...m.groups, status: Number(m.groups.status)
})));

const email = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;
console.log(['a@b.co', 'no-at.com', 'x@y', 'sp ace@a.io'].map((e) => `${e}:${email.test(e)}`).join('
'));

const n = 1234567.891;
console.log(String(Math.trunc(n)).replace(/\B(?=(\d{3})+(?!\d))/g, ','), n.toLocaleString('en-US'),
n.toLocaleString('de-DE'));

const tpl = 'Hi {name}, your code is {code}. {unknown}';
const data = { name: 'Ana', code: 'X9' };
console.log(tpl.replace(/\{(\w+)\}/g, (m, key) => data[key] ?? m));
```

Output:

```
┌─────────┬─────────────┬──────────────────────┬────────┬──────────────┬────────┬─────────┐
│ (index) │ ip          │ time                 │ method │ path         │ status │ bytes   │
├─────────┼─────────────┼──────────────────────┼────────┼──────────────┼────────┼─────────┤
│ 0       │ '127.0.0.1' │ '10/Oct/2024:13:55:36' │ 'GET'  │ '/index.html' │ 200    │ '2326' │
│ 1       │ '10.0.0.5'  │ '10/Oct/2024:13:56:01' │ 'POST' │ '/api/login'  │ 401    │ '87'   │
└─────────┴─────────────┴──────────────────────┴────────┴──────────────┴────────┴─────────┘
a@b.co:true no-at.com:false x@y:false sp ace@a.io:false
1,234,567 1,234,567.891 1.234.567,891
Hi Ana, your code is X9. {unknown}
```

1. **Log parsing**: `gm` flags + named groups + `matchAll` turn each line into an object. Real log pipelines (systems that collect, parse and ship logs from many servers — e.g. Logstash "grok" patterns, Fluent Bit parsers) are built on exactly this idea.
2. **Email**: a *pragmatic* check ("something@something.tld", no spaces; the TLD — top-level domain — is the last part like `.com`). Fully validating RFC 5322 (the official email-format standard) addresses with a regex is impractical; the only real test is sending a confirmation email. HTML's `<input type="email">` uses a similar simplified rule.
3. **Thousands separators**: the lookahead `(?=(\d{3})+(?!\d))` finds positions followed by groups of three digits — a neat trick, but `toLocaleString` / `Intl.NumberFormat` is correct for every locale.
4. **Templating**: a replacer function looks each `{key}` up and leaves unknown keys untouched.

Trade-offs: regex or string methods?

Situation	Prefer
Fixed substring test / replace	`includes`, `startsWith`, `replaceAll` — clearer and faster
Patterns (digits, optional parts, alternatives)	Regex
Structured formats (HTML, JSON, CSV with quotes, URLs)	A real parser: `DOMParser`, `JSON.parse`, a CSV library, `new URL()`
Human-language text (words, characters)	`Intl.Segmenter`, `\p{…}` with `u` / `v`

When not to use it

Avoid giant "do everything" regexes nobody can read; split the job into steps or write a small parser. Never run a user-supplied pattern without a timeout — a limit after which the work is abandoned — (or a linear-time engine such as RE2, whose running time only grows in proportion to the input) — catastrophic backtracking (the engine trying an explosive number of ways to match, see the interview section below) can hang a Node server, which is how several real outages (for example Cloudflare's July 2019 outage) happened.

Common mistakes

- Forgetting anchors in validation (`/\d{5}/.test('abc123456')` is `true`).
- Forgetting `g` in `replace`, or using `matchAll` / `replaceAll` with a non-global regex (both throw a `TypeError`).
- Reusing a `/g` regex with `test` (the `lastIndex` trap).
- Building `new RegExp(userText)` without `RegExp.escape`.
- Writing `'\d'` instead of `'\\d'` in the `RegExp` constructor string.
- Expecting `\w` to match `é` or `ñ` — it is ASCII-only; use `\p{L}` with `u`.

Real-world examples

- **Routers**: Express (the most popular Node web-server framework) and many front-end routers compile `/users/:id` into a regex with a capture group.
- **Linters and formatters** use regexes for simple rules; editors use the `d` flag's indices for highlighting.
- **Form validation** with the HTML `pattern` attribute is a regex (evaluated with the `v` flag in modern browsers).
- **Log processing** and **search tools** (ripgrep, VS Code search) are built on regex engines.

In the interview

> **30-second answer:** "A regex is a pattern object — literal `/…/flags` or `new RegExp`. I use `test` for yes/no, `matchAll` with named groups to extract, and `replace` with a callback to transform. Key gotchas: anchors for validation, `g` needed for global replace, stateful `lastIndex` with `g`, greedy vs lazy quantifiers, and escaping user input with `RegExp.escape`."

- *What's the difference between `match` and `matchAll`?* `match` with `g` returns only the strings (or `null`); `matchAll` returns an iterator of full match objects with groups and indexes.
- *How do you match a word in any language?* `/\p{L}+/u`.
- *What is catastrophic backtracking?* Nested quantifiers like `(a+)+$` try exponentially many ways to split the input when it almost matches; avoid nested unbounded quantifiers.

Self-check

1. What does `/^\d{3}-\d{4}$/` match? → Exactly three digits, a dash, four digits — e.g. `555-1234`.
2. `'aaa'.replace(/a/, 'b')`? → `'baa'`.
3. How do you refer to named group `year` in a replacement string? → `$<year>`.
4. Why is `/<.+>/` wrong for matching one tag? → Greedy; it stretches to the last `>`. Use `/<[^>]+>/`.
5. Which ES2025 method safely escapes user input for a regex? → `RegExp.escape`.

Chapter TL;DR

- Strings are **immutable UTF-16 primitives**; create them with `'…'`, `"…"` (identical) or backticks (interpolation, multi-line, tags). Escape with `\` (`\n`, `\t`, `\\`, `\'`, `\u{…}`).
- Index with `[i]`, `charAt`, or `at(-1)`; out-of-range brackets give `undefined`. Assigning to an index never changes a string (throws in strict mode).
- `+` concatenates as soon as one side is a string; other arithmetic operators convert to numbers. Avoid `new String` — wrappers are objects compared by reference; primitives get methods via auto-boxing.
- Core methods: `includes`/`startsWith`/`endsWith`/`indexOf`, `slice` (negatives OK), `toUpperCase`/`toLowerCase`, `trim*`, `padStart`/`padEnd`, `repeat`, `split`/`join`, `replace` (first only) / `replaceAll`, `localeCompare`/`Intl.Collator` for human ordering.
- `length` counts code units: emoji are 2. Iterate with `for…of`/spread for code points; use `Intl.Segmenter` for visible characters; `normalize('NFC')` before comparing; `isWellFormed`/`toWellFormed` for lone surrogates.
- Regexes: literals or `new RegExp`; classes, quantifiers (greedy by default, `?` for lazy), anchors, groups (numbered, named, non-capturing), lookaround, backreferences; flags `g i m s u v y d`.
- Methods: `test`, `match` (null on no match), `matchAll` (needs `g`), `replace` with `$1`/`$<name>` or a callback, `split` by regex. Beware `lastIndex` with `/g`.

- ES2024–2025 additions: `v` flag, `RegExp.escape`, pattern modifiers, duplicate named groups — all in Node 24. Prefer string methods for fixed text and real parsers for structured formats.